

Online Voting System: Project Report

Arnab Ghosh, 2322019042

Fardin Ahmed, 2323806642

Department of Electrical and Computer Engineering,

North South University.

Faculty: MsRB

Date of Submission: 02.12.2024

Abstract- Online Voting System is a relatively new technology which allows users to vote online instead of going to far polling centers. The report shows that it is a multi-threaded application designed to facilitate smooth and secure voting procedures for both polls and surveys. Users can cast votes concurrently, simulating real-time scenarios. The results of the votes are stored in separate files for polls, surveys and open-ended responses.

Keywords- Online Voting System, multi-threading, synchronization, castVote(), displayResults().

1. Classes:

This report will give detail explanation about the functions and technicalities of the five classes: Voting, Polls, Surveys, VoteThread and OnlineVotingSystem (main). Each of these classes are integral as to how the program runs and functions.

A. *Voting class:* The Voting class is an abstract class that declares two abstract methods “castVotes” and “displayResults”. which will later be implemented in the subclasses “Polls” and “Surveys”.

- 1) *Protected:* The ‘protected’ modifier is used for declaring ‘question’ and ‘totalvotes’ in this class as it allows access to the subclasses of Voting

and protects the integrity of the program.

- 2) *Constructor:* The constructor Voting is created with question as it’s parameter. Any question can be asked by the user because of this constructor.
 - 3) *Abstract methods:* The abstract methods ‘castVotes’ and ‘displayResults’ are created in this class but without implementation. They will be implemented later in the subclasses.
 - 4) *SaveToFile:* A synchronized method with data and filename as parameters is created with a try-catch block where if there is no IOException, the data is written into the file.
 - 5) *Synchronization:* Using synchronization prevents clashing of multiple threads and causing inconsistent data. This allows only one thread to execute at a time.
 - 6) *FileWriter :* Enables use of FileWriter class which has to be imported from java.io package and it is a part of java’s standard input-output library. FileWriter allows the user to write into the files.
 - 7) *IOException:* If the file doesn’t exist or there’s no disk space, the program will throw an IOException. It has to be imported from the java.io package which is a part of java’s standard input-output library.
- B. *Polls class:* The Polls class is a subclass of Voting.

- 1) *Declaring polling options:*
Four options of 'int' datatype with 'private' (as it is only needed to be accessed from the same class) visibility modifier are declared. The options are: a_votes, b_votes, c_votes and d_votes.
 - 2) *Constructor:* 'Polls' constructor is created with question as modifier. It calls question from the superclass Voting using 'super'.
 - 3) *@Override:* The method is overridden to implement 'castVote' and 'displayResults' from the superclass Voting.
 - 4) *'castVote' method:* It is a public synchronized (so the threads don't clash) method with "option" as the parameter. An if-else loop is used to count votes for each of the four options. After each vote is taken, 'totalvotes' is incremented.
 - 5) *SaveToFile:* Each poll vote is saved to the file "pollsresults.txt".
 - 6) *'displayResults()' method:*
This synchronized public method prints the amount of votes in each option and the total votes.
- C. *Surveys class:* The Surveys class is a subclass of Voting.
- 1) *Declaring 'votes' and 'options' arrays:* It allows for clean code and available number of answer options. No need to declare multiple times.
 - 2) *Constructor:* The constructor 'Surveys' is created with 'question' and 'options' as parameters. This helps set the question by calling question from Voting. The number of options are set. Votes are counted to each option.
 - 3) *@Override:* The method is overridden to implement 'castVote' and 'displayResults' from the superclass Voting.
 - 4) *'castVote' method:* A public synchronized method with option as parameter is created which counts votes if it is a valid option and saves to file. The total votes is also counted.
 - 5) *validOption:* 'validOption' of Boolean datatype is initially declared false. But if the input from user is a valid option, then 'validOption' becomes true and a vote is counted. Otherwise, "Please enter a valid option" is printed.
 - 6) *SaveToFile:* If 'validOption' = true, 'totalvotes' is incremented each time and the results are saved to the file "surveysresults.txt".
 - 7) *displayResults() method:* The survey results for each option and the total votes counted are printed.
- D. *VoteThread class:* The VoteThread class extends the in-built 'Thread' class.
- 1) *Declaring 'votingSystem' and 'voteOption':* An object 'votingSystem' is created of 'Voting' type. 'voteOption' is created of datatype String. Both have private visibility modifiers.
 - 2) *Constructor:* A 'VoteThread' constructor is created with votingSystem and votingOption as parameters. This creates the 'VoteThread' object by setting the values of votingSystem and votingOption.
 - 3) *@Override:* The 'Thread' class has an in-built method 'run()'. So to implement it in the 'VoteThread' class, the run() method needs to be overridden.

- 4) *Run() method:*
votingSystem.castVotes (voteOption). This implementation of the run method allows multiple threads to concurrently call the 'castVotes()' method, handling votes in parallel for various options.

E. *OnlineVotingSystem class:* This is the main class of the whole program.

- 1) *Scanner:* This is imported from the java.util package. It is used to take input for values.
- 2) *Infinite Loop:* 'while(loop)' is used to keep the loop running unless "stop" or "exit" is taken as input.
- 3) *votingSystem:* 'votingSystem' object of 'Voting' type is declared with null value. The implementation will be assigned depending on the choice of the user. If choice is 1, then the value is declared as "votingSystem = new Polls(question)". If choice is 2, the value is declared as "votingSystem = new Surveys(question, options)".
- 4) *choice:* The user is given a choice when they open the Online Voting System program. If the user chooses '1', a poll will be created. If '2' is chosen, a survey will be created. And if 'stop' is chosen, the program will exit.
- 5) *Choice 1 (Polls):* When the user inputs 1 as their choice, they can create a polling question. After that, voters can vote in one of four options. The options are: a (Hard Agree), b (Agree), c (Disagree), d (Hard Disagree). If 'a', 'b', 'c' or 'd' is input, a vote is counted to each of these options. Voters can keep

voting until 'exit' is input to stop the polling.

- 6) *Choice 2 (Surveys):* When the user inputs 2 as their choice, they can create a survey question. The numbers of options and the options themselves can be set by the user. To count a vote, the voters have to input the specific option. The votes will continue to be taken until an 'exit' input is taken to stop the survey. After the survey, the user is given a choice to ask an open-ended question. If the user chooses 'no', the survey will end. If the user chooses 'yes', an extra question is asked to the voter. The voter is asked to enter a suggestion or answer. Once they are done, they can input 'exit' to end the survey. The open ended response of the voter is saved to the file 'openEndedResponse.txt'.
- 7) *Choice 'stop':* The program will end.
- 8) *Invalid choice:* If any input other than '1', '2' or 'stop', is chosen, "Invalid choice. Please enter '1', '2' or 'stop' to exit." is printed.
- 9) *Creating threads:* "Thread VoteThread = new VoteThread (votingSystem, vote)" creates a thread for each vote for each option. VoteThread.start() starts the thread.
- 10) *votingSystem.displayResults():* Results of votes of polls and surveys are printed along with the total votes taken.

2. Implementation of multithreading to handle concurrent votes: Each vote is processed in it's own thread using the 'VoteThread' class. Threads call the 'castVotes()' method concurrently, allowing multiple users to vote simultaneously. After threads finish, the

main thread compiles and displays results.

3. Implementing File I/O to store voting results: The “SaveToFile()” method in Voting class appends voting data to a file. It uses ‘FileWriter’ with the ‘append’ mode set to ‘True’. For polls, each vote is saved as a line in “pollsresults.txt”. For surveys, each custom vote options are saved in “surveysresults.txt”. Open ended responses are saved in “openEndedResponse.txt”.